

Thinning state vectors, and what the archive actually contains

Why the 5 s decimation rule is not the problem, and what is

EUROCONTROL Performance Review Unit

2026-08-09

Table of contents

Summary	1
1 What was measured	2
2 Result 1: the two rules are indistinguishable	2
3 Result 2: the effect exists, in the right place, at the wrong size	3
4 Result 3: what the row counts were hiding	4
4.1 The asymmetry between the two ends of a track	4
5 Result 4: the end-to-end test agrees	5
6 What to do with this	6
7 Limitations	6
8 Reproducing	7

Summary

The ingest thins OpenSky state vectors to roughly one sample every 5 s with a modulo filter, `event_time % 5 == 0`. That is a **fixed-phase** sampler: it keeps a row only if one exists at the single second per window congruent to zero. Where reception is intermittent — low altitude, long range, the edge of the receiver network — the phase of the data is arbitrary, so the filter should delete far more than it is supposed to. Since version 4 of the [ADEP/ADES study](#) found arrivals much harder than departures, and arrivals are exactly the flights whose evidence sits in that sparse region, the filter was a plausible cause.

It is not. Measured on three hours of raw archive, a correct bin-based sampler keeps $1.002\times$ the rows of the modulo filter — 17,467 rows out of 7,820,363, or 0.22%. The hypothesis fails because its premise does: the OpenSky `state_vectors` table is not a record of receptions but a **complete 1 Hz grid**, with the last known position carried forward when nothing is heard.

The experiment then found what the row counts were hiding. Positions in that grid are often old, and the staleness is wildly asymmetric between the two ends of a track:

	median position age
first row of an aircraft	0 s
last row of an aircraft	190 s

A departure’s first fix is fresh by construction. An arrival’s last fix, at the median, carries a position measured **190 s earlier** — at approach speed, five to eight nautical miles back down the approach. 55.10% of last fixes are more than 30 s stale.

That is not a bug to fix in the decimation rule. It is a measurement of how early reception ends, and it quantifies the asymmetry version 4 could only assert. It also suggests something the detection can act on, because `last_pos_update` is already ingested and read by nothing.

1 What was measured

Three hourly partitions of `s3a://opensky-hdfs-backup/tables_v4/state_vectors`, 2025-06-05 06:00–09:00 UTC, filtered to the OPDI bounding box and nothing else: **39,139,623 rows**. Both rules were then applied to identical input, with a cold read from S3 each time so the comparison includes the I/O the production job actually pays.

Modulo (current). Keep the row where `event_time % 5 == 0`. A narrow filter; no shuffle.

Bucket (proposed). Bin time into `floor(t/5)` and keep one row per non-empty bin, via `max` over a struct with `event_time` first — a hash aggregate with map-side partial aggregation, native Spark throughout.

2 Result 1: the two rules are indistinguishable

Table 1: Both rules on 39,139,623 rows, three hours of archive. The S3 read is common to both, so the difference net of it is the cost of the shuffle.

Rule	Rows	Wall clock	Net of the read
bbox only (baseline)	39,139,623	26.4 s	—
modulo (current)	7,820,363	26.8 s	+0.4 s
bucket (proposed)	7,837,830	30.2 s	+3.8 s

The bucket rule keeps $1.002\times$ the rows. The modulo filter retains $1/5.005$ of its input — within a whisker of the exactly one-in-five it would keep if every second carried a row.

The density measurement confirms why:

Table 2: Fraction of each minute’s 60 seconds carrying a row, per aircraft-minute. The archive is a dense grid, not a log of receptions.

Rows per aircraft-minute	Aircraft-minutes	Share	Mean density
<0.1 (sparse)	1,513	0.23%	0.051
0.1-0.3	3,765	0.57%	0.192
0.3-0.5	3,877	0.59%	0.392
0.5-0.7	3,959	0.60%	0.592
0.7-0.9	4,055	0.61%	0.793
0.9-1.0 (dense)	644,745	97.41%	1.000

97.41% of aircraft-minutes are at least 90% populated. The sparse-coverage model that motivated this experiment applies to a few percent of the data.

! The arithmetic was right; the premise was wrong

The predicted gain was 2–5 $\times$, derived from a model in which a second carries a row with probability $p < 1$. Measured p is 1.0. Everything downstream of that assumption — retention ratios, endpoint-lag formulae, loss probabilities — described a feed that does not exist. The model was not tested before it was believed; this experiment is what testing it looks like.

3 Result 2: the effect exists, in the right place, at the wrong size

The extra rows are not distributed evenly. They concentrate exactly where the hypothesis said they would.

Table 3: Rows the bucket rule rescues, by altitude. *Rescue rate* is extra rows over retained rows; *over-rep* is the share of rescued rows against the share of all rows, so above 1 means they concentrate there.

Band (ft)	Rows	Extra	Rescue rate	% of rows	% of extra	Over-rep
(null)	748,206	3,197	0.43%	9.57%	18.30%	1.91x
0-2,000	571,978	2,811	0.49%	7.31%	16.09%	2.20x
2,000-5,000	570,432	1,270	0.22%	7.29%	7.27%	1.00x
5,000-10,000	520,655	979	0.19%	6.66%	5.60%	0.84x
10,000-20,000	786,552	1,372	0.17%	10.06%	7.85%	0.78x
20,000-30,000	813,690	1,264	0.16%	10.40%	7.24%	0.70x
30,000+	3,808,850	6,574	0.17%	48.70%	37.64%	0.77x

The rescue rate runs monotonically from **0.49% at 0–2,000 ft** down to **0.16% at 20–30,000 ft** — a 3.2 $\times$ gradient from the surface to cruise. The lowest band is 2.20 $\times$ over-represented among the rescued rows and the null-altitude band — largely surface traffic — 1.91 $\times$. Every band above 5,000 ft is at or below parity.

So the mechanism is real and it is altitude-dependent in the predicted direction. It is also **0.22% of the data**, which cannot explain an eleven-point coverage gap. Direction confirmed, magnitude irrelevant.

The endpoint check agrees: of 8,220 aircraft, modulo erased 3 (0.04%) entirely, and its last retained row sits a median of 4 s behind the true last row — about a quarter of a nautical mile.

On this evidence the decimation rule should not be changed. It costs +13% in wall clock, it would break `track_id` continuity with every published dataset, and it buys 0.22% more rows.

4 Result 3: what the row counts were hiding

The archive fills every second, but it fills them by **carrying the last known position forward**. `lastPosUpdate` records when the position was actually measured, so `age = event_time - lastPosUpdate` is how old the position in a row is. A row can be present and carry nothing new.

Table 4: Age of the position carried by each row, by altitude. The mean is not shown: the distribution is bimodal – mostly fresh, with a heavy stale tail – and the mean describes neither part.

Band (ft)	Rows	Median age	p90	p99	> 5 s	> 30 s
(null)	3,743,758	6 s	186 s	296 s	51.40%	32.30%
0-2,000	2,861,939	2 s	203 s	295 s	43.40%	33.20%
2,000-5,000	2,854,850	0 s	28 s	266 s	15.60%	10.00%
5,000-10,000	2,605,554	0 s	5 s	293 s	10.20%	7.30%
10,000-20,000	3,936,633	0 s	2 s	329 s	8.20%	6.00%
20,000-30,000	4,072,494	0 s	1 s	394 s	5.90%	4.30%
30,000+	19,064,395	0 s	2 s	868 s	8.20%	5.80%

At cruise, positions are fresh: 4.30% of rows above 20,000 ft are more than 30 s old. **Below 2,000 ft that rises to 33.20%, and on the surface to 32.30%.** A third of all low-altitude rows carry a position measured more than half a minute earlier.

This is the user-visible form of intermittent low-altitude coverage. It does not appear as missing rows because OpenSky fills the gap for us.

4.1 The asymmetry between the two ends of a track

Table 5: Position age at the two ends of each aircraft’s presence in the window. This is the sharpest asymmetry in the study.

Endpoint	N	Median age	p90	p99	> 30 s
first row of an aircraft	8,220	0 s	2 s	9 s	0.60%
last row of an aircraft	8,218	190 s	305 s	1263 s	55.10%

A departure’s first fix is fresh by construction — an aircraft enters the table when it is first heard, so there is no earlier position to carry forward. Median age 0 s, 0.60% older than 30 s.

An arrival’s last fix is stale by construction — the aircraft stays in the table after it stops being heard, padded with its last known position until it times out. Median age **190 s**, and **55.10%** older than 30 s.

At 150 kt on final approach, 190 s is about 8 NM.

⚠ What this does and does not mean

It does **not** mean the endpoint position is wrong. The carried-forward position is the last position genuinely measured, which is still the best available estimate of where the track ended. Deduplicating the stale rows would not move it.

What it means is that **reception ends a median of 190 s before the aircraft leaves the data**, and the aircraft keeps flying through that interval. Version 4 asserted that an arrival’s last sample is “where reception died rather than where the flight ended”; this puts a number on it, and the number is minutes rather than seconds.

It also means three things downstream are measurably wrong, none of them detection: **last_seen** event times are late by the same interval; gap-based track splitting sees padded gaps rather than real ones; and any derivative computed across carried-forward rows is diluted by runs of zero change.

5 Result 4: the end-to-end test agrees

Rows are a proxy. What matters is whether detection changes, so the whole chain was re-run on bucket-sampled input — ingest, tracks, endpoint candidates — and scored against the same ground truth at the same operating point. One day, 2025-06-05, both arms; **endpoint** mode at 30 NM, 15,000 ft and a 10 NM scheduled-service penalty, applied to **both** roles.

Table 6: Detection on modulo- against bucket-sampled input, 32,395 ground-truth flights on 2025-06-05. **endpoint** at 30 NM / 15,000 ft / 10 NM, both roles. Each cell is modulo then bucket.

Movements	Coverage	d cov	Accuracy	d acc	Overall	d ovr
departures	81.22% -> 81.19%	-0.03 pp	98.15% -> 98.14%	-0.01 pp	79.71% -> 79.68%	-0.03 pp
arrivals	69.20% -> 69.46%	+0.26 pp	95.70% -> 95.59%	-0.12 pp	66.23% -> 66.40%	+0.17 pp

Table 7: The same comparison in flights.

Movements	Answered	Correct	Wrong
departures	26,310 -> 26,301 (-9)	25,822 -> 25,811 (-11)	488 -> 490 (+2)
arrivals	22,417 -> 22,502 (+85)	21,454 -> 21,509 (+55)	963 -> 993 (+30)

The candidate cache grew from 2,586,372 to 2,598,185 rows, +0.46% — about twice the 0.22% gain in the state vectors themselves. That is the one place the rescued rows are worth more than average rows: they sit at track boundaries, and boundaries are exactly what the candidate stage extracts.

Departures do not move: -11 correct labels, which is noise. **Arrivals move slightly and in the predicted direction:** +85 answered and +55 correct, for +30 wrong. That is +0.17 pp of arrival overall correctness.

The asymmetry is the theory working: the bucket rule keeps the *last* row of each bin, so a track’s final observation survives exactly, while the modulo rule lands a median of 4 s short of it. Departures have nothing to gain from that — their first fix is already well located — and arrivals have a little.

A little is the operative word. Fifty-five additional correct arrivals in a day, against thirty additional errors, for a change that rewrites `track_id` for every published row. The end-to-end test confirms the row-count prediction rather than overturning it.

6 What to do with this

Do not change the decimation rule. 0.22% of rows, at +13% wall clock and the loss of `track_id` continuity, is not a trade worth making.

Use position age as an abstention signal. This is the actionable finding. `last_pos_update` is already ingested, already carried into `osn_tracks` (`tracks.py:455`), and read by nothing. A track whose final fix has an age of 300 s has an endpoint that is several miles from where the flight actually ended, and the detection has no way to know it. Age at the endpoint is a direct, free measure of how much to trust that endpoint — and unlike the height threshold of the ADEP/ADES version 4 rule, it measures the thing that actually goes wrong.

That gives a candidate rule worth testing against the version 4 operating point: abstain when the endpoint’s position age exceeds a threshold, instead of, or in addition to, when its height exceeds one. It is a one-column change to the candidate cache.

Consider dropping stale rows at ingest. Not to improve detection, but for storage and for honesty: 33.20% of low-altitude rows are repeats of a position already recorded. Removing them would shrink the store and would make gap-based track splitting see the gaps that are really there. It would also change `track_id`, so it belongs in the same versioned re-ingest as any other ingest change.

7 Limitations

- **Three hours, one day, one bounding box.** 2025-06-05 06:00–09:00 UTC is a busy European morning; a night sample or a different region may differ.
- **“Last row of an aircraft” is not “last row of a flight.”** The window’s end truncates aircraft still airborne, and an aircraft leaving the bounding box exits with a fresh position. Both dilute the staleness figure, so the true landing-endpoint staleness is likely worse than reported, not better.
- **Position age is not position error.** A stale position is exactly right for the moment it was measured. The error it causes downstream depends on what the aircraft did next, which is unobserved by construction.
- **The bucket rule keeps the last row per bin, not the freshest.** Given the measured penalty of under 2 s of staleness this makes no practical difference, but a rule optimising freshness would use `min_by(age)`.

8 Reproducing

```
python benchmarks/decimation_experiment.py \  
    --date 2025-06-05 --start-hour 6 --hours 3 --results-dir <dir>
```

Rendering this page needs no credentials, no database and no cluster.